

InterOpera — Senior Software Engineer Take-Home

Issued by: InterOpera Engineering **Estimated effort:** 24–32 hours over 7 days **Submission deadline:** 7 days from receipt

We are not looking for polish — we are looking for an engineer who can read a set of hard requirements, **derive** the right architecture, and build a system that holds up to an audit.

Background

Asset managers and fund administrators run portfolios against a book of rules — investment guidelines, allocation limits, risk thresholds, concentration caps — and must produce **periodic regulatory reports** that state, for every rule, whether the current portfolio is inside or outside its limit, by how much, and where each figure came from.

Today this is done by hand. An analyst reads the guidelines document, pulls the relevant numbers out of a holdings snapshot, computes each allocation, utilization, and breach flag in a spreadsheet, and types the results into a report template. The work is slow, easy to get subtly wrong, and — most importantly — **hard to defend in an audit**. When an examiner points at a number and asks "where did this come from, and who computed it?", the answer is often a formula buried three tabs deep in someone's working file.

These firms operate under regulatory oversight (the sample fund here is SGD-denominated and MAS-supervised). For a report to be acceptable to an audit examiner, it is not enough for the numbers to be *correct* — the firm must be able to **prove** how each number was produced, from which source passages, by which method, and that nothing in the chain was fabricated or quietly edited.

Your task: design and build a working system that produces such a report from the materials we provide, satisfying the hard constraints below. We give you one firm's expected output to reconcile against, and a second firm that runs the same portfolio under different house conventions. How you architect the system to meet the constraints is the substance of what we are evaluating — we state the *requirements*, not the design.

The hard constraints

These are non-negotiable. They are the requirements an audit examiner and a second client firm impose on the system. Read them carefully before you design anything — they are the spine of this assignment, and everything we evaluate traces back to them.

1. Every reported number must be **reproducible**: running the system twice on the same inputs yields identical figures.
2. Every reported number must be **traceable**: an auditor can follow each figure to the exact source passage(s) it was derived from, **through the knowledge graph** (`figure` → `graph path` → `source chunk`).
3. An auditor must be able to verify that **no figure was produced or altered by the language model**. The LLM may write narrative commentary only.
4. The system must **reproduce Firm A's answer key** (exact, or within a stated tolerance you justify).
5. The system must be **reconfigurable to Firm B's method** (`firm_B_brief.md`) to reproduce Firm B's answer key — **without changing engine code**.

A note on constraint 2: a knowledge graph is a **required** part of your solution. The portfolio rules are a web of relationships — a limit belongs to an asset class, a risk metric has a threshold and a breach action with an owner, an issuer rolls up to a parent. Each reported figure must be traceable along an explicit path through that graph back to the source passage that justifies it. The other four constraints describe *properties your system must have*; how you achieve them is yours to derive and defend.

Materials provided

All sample materials describe a fictional fund — **Meridian Fixed Income Fund** — and are entirely synthetic. Find them in `sample_docs/`.

FILE	WHAT IT IS
<code>sample_fund_guidelines.pdf</code>	The rules source: allocation limits, risk metrics and their thresholds, concentration caps, liquidity floors, breach actions, owners, and retention requirements. This is the authoritative statement of what every limit is.
<code>sample_holdings.csv</code>	A period-end portfolio snapshot : one row per instrument with asset class, issuer, issuer type, parent issuer, credit rating, market value (SGD), and modified duration. This is the data you compute over .
<code>report_template.xlsx</code>	The blank target output. Your system populates it.
<code>firm_A_answer_key.xlsx</code>	The expected report as Firm A computes it — the ground-truth figures your output must reconcile to.
<code>firm_B_brief.md</code>	The same fund and the same holdings, administered by a second firm whose house conventions differ on a few computations. The reconfiguration test.

Every figure in both firms' answer keys is hand-derivable from `sample_holdings.csv` plus `sample_fund_guidelines.pdf`. You may add small mock documents if your design requires them — if you do, say why.

What we evaluate (in priority order)

1. **Do you satisfy the five constraints?** This is the assignment. A system that is elegant but lets the model produce numbers, or that can't reconfigure to Firm B without a code edit, has missed the point.
2. **Is traceability real, and is it through the graph?** We will pick a figure and follow it [figure → graph path → source passage](#). If the graph is built but the numbers are computed somewhere it isn't involved, that is a fail.
3. **Does it run?** We execute your submission. If it doesn't start, we cannot evaluate the implementation phases.
4. **Graph-modeling and computation judgment.** Did you model the right entities and relationships for this domain, and compute each figure soundly?

We will, concretely: (1) run the system twice and diff the numbers; (2) trace one figure end-to-end through the graph to its source; (3) switch from Firm A's configuration to Firm B's and confirm the figures change with **no code edit**.

Deliverables

The assignment is 100 points across five phases. Each phase is framed as an *outcome against the constraints above* — we describe what must be true of your system, not how to build it.

Phase 1 — Business understanding, architecture & RFC (20 pts)

- An **AS-IS / TO-BE** flow of the reporting process. The TO-BE must show where the system acts autonomously and where a human reviews or approves — **including a gate where a human verifies the extracted graph** before its contents are trusted in a report (entity/relationship extraction is error-prone). For each gate, state the criterion that decides auto-pass vs. human review.
- An explicit, stated **boundary between what may be generated by the language model and what must be produced deterministically**. This boundary is the heart of constraint 3 — make it concrete, not aspirational.
- An **audit event catalogue** — every event the system records to satisfy an examiner, as a table ([Event](#) , [Trigger](#) , [Data Captured](#) , [Retention](#)). At minimum it must cover graph construction, figure computation, reconciliation against an answer key, and a configuration change.
- An **RFC** (a short technical design memo, prose not code) that **derives** the architecture from the constraints and defends the key decisions. It must explain: how the system **structurally guarantees the language model cannot be the source of any reported number** (constraint 3); how a figure is traced through the graph to its source (constraint 2); how a firm's method is expressed and switched (constraint 5); and how output is reconciled to an answer key (constraint 4). We are reading for *why*, not just *what*.

Deliverables: [docs/01_flow_and_audit_events.md](#) , [docs/02_architecture.\[md|png|pdf\]](#) , [docs/03_rfc.md](#)

Phase 2 — Knowledge graph (15 pts)

Ingest **both** the guidelines (limits, risk metrics, thresholds, breach actions, owners, retention) **and** the holdings snapshot (positions) into **one** graph.

- Model the right entities and relationships for this domain (asset classes and their limits; risk metrics, their thresholds, and breach actions with owners; issuers and their parent rollups; positions and the asset classes / issuers they belong to).
- **Every node and every edge carries provenance** — the source document, page, chunk, ingestion time, and an `extraction_confidence`.
- The graph must be **multi-hop queryable** — e.g., you can answer "what is the breach action if portfolio duration exceeds its limit, and who is notified?" by traversing the graph rather than re-reading the document.

Phase 3 — Computed, traceable figures (30 pts) ★

This is the core of the assignment.

- Compute each report figure by **traversing the graph** — allocation % per asset class vs. its limit, with utilization and a breach flag; aggregate non-IG exposure vs. its cap; single-issuer and group concentration vs. their caps; liquidity ratio vs. its floor; portfolio-level duration / DV01-style figures.
- The figures must come **only from your computation layer**. The language model must not be in the path that produces, rounds, or alters any number (constraint 3). If it writes narrative, that narrative may not introduce a number absent from the computed output.
- The computation must be **deterministic** (constraint 1): re-running on the same inputs yields byte-identical figures.
- **Each figure emits its value, its graph path, and its citation.** A figure that cannot be traced (`figure → graph path → source`) must be returned as an **error, not silently emitted**. The output shape we expect per figure:

```
{
  "figure": "aggregate_non_ig_exposure",
  "value": "15.0%",
  "status": "OK",
  "limit": "max 20%",
  "graph_path": "(AssetClass:high_yield)-[:CONTRIBUTES_T0]->(Aggregate:non_ig)<-[:CONTRIBUTES_T0]-(AssetClass:structured_credit)",
  "citation": {
    "source_doc": "sample_fund_guidelines.pdf",
    "page": 4,
    "chunk_id": "chunk_9c1a",
    "passage_summary": "Section 4.2 – aggregate non-investment-grade exposure cap"
  }
}
```

- Your output must **reconcile to** `firm_A_answer_key.xlsx` — exactly, or within a tolerance you state and justify (constraint 4).

Phase 4 — Reconfiguration to Firm B (20 pts) ★

`firm_B_brief.md` describes the same fund and the same holdings, but Firm B computes several figures by a different method. Your system must reproduce **both** firms' answer keys.

- Switching from Firm A to Firm B must **not require an engine-code edit** (constraint 5).

- A submission with Firm A baked into the computation logic cannot reproduce Firm B without an edit — that is exactly what this phase catches. How each firm's method is expressed and kept switchable is your design choice.

Phase 5 — Reconciliation + no-LLM-numbers evaluation (15 pts)

A script that reports, in readable form (table or JSON):

- **Per-figure reconciliation** vs. the answer key — pass/fail and the delta for each figure.
- A **traceability check** — every figure resolves `figure → graph path → source`.
- A **firewall check** that proves the narrative layer introduces **no number that is not present in the computed output** (constraint 3, verified rather than asserted).

Bonus (+5, capped; only if Phases 3–5 are complete)

- A **reconciliation / replay viewer**: given a figure, show its graph path, its source, its delta vs. the answer key, and which configuration rule produced it (+2–4).
- A small **configuration mini-DSL with live preview** for expressing a firm's method (+2–3).
- **Global/local retrieval** for the narrative layer (+1–2).

The append-only audit requirement

The system must keep a **persistent, append-only audit log** of its run: graph construction, figure computation, reconciliation, configuration change, and export. No row may be updated or deleted after insertion — demonstrate this in code (e.g., no `UPDATE` / `DELETE` path, or a DB constraint/trigger). This is the record an examiner replays to reconstruct exactly how a report was produced.

Submission format

- Provide a **runnable repository** with a clear README. It must start with a single documented command — `docker compose up`, or `pip install -r requirements.txt && python <entrypoint>`. If it does not start, Phases 2–5 cannot be evaluated.
- Organize the code into modules with clear boundaries. **We do not prescribe the structure** — how you arrange the system is part of what we are evaluating.
- We must be able to produce **both** Firm A's and Firm B's reports without you editing code between the two runs. How you make the two firms switchable is your design decision.
- Include the following documentation deliverables under `docs/`: the flows + audit-event catalogue, the architecture diagram, and the RFC (see Phase 1).
- Include the provided `sample_docs/` files in the repository.

LLM API: use any frontier API (Anthropic Claude, OpenAI, Google Gemini) or a self-hosted open model. If you self-host, document the rationale — we treat it as a positive signal. Use your own key;

Language: all documentation and code comments in English.

Notes on scope

This is a one-week assignment, not a production system. We expect a working system that proves the constraints hold — not polished software.

- **Error handling:** cover the happy path plus one or two obvious failure modes (e.g., an extracted entity the system can't confidently resolve, or a figure that can't be traced and is therefore returned as an error). Exhaustive exception handling is not expected.
- **Security:** no production-grade auth or secrets management needed. Note what you would add for production.
- **Scale:** design for the sample materials provided. **Graph and computation quality over size** — a well-modeled graph of this fund and a correct, traceable set of figures beats a large noisy one.
- **Reconfiguring to Firm B must not require editing code** (constraint 5) — how you achieve that is your design choice.
- The bonus items are optional and evaluated only if Phases 3–5 are complete.

What we care about: do the five constraints hold; is traceability real and through the graph; does it run; and does your modeling and computation show sound judgment.